

Financial Literacy for Students

Porter Clevidence

Antonio Duran-Ramos

Parth Gajjar

Arpit Vora

May 6, 2025

List of Contents

List of Figures	1
List of Tables	2
Project Plan	3
Introduction	3
Team Roles and Responsibilities	3
Methods and Techniques	3
Data	3
Data Description and Analysis	4
Dataset Statistics	4
Text Processing	5
Surface Level Normalization Definition	5
Tokenizer Definition	5
Stopping Definition	5
Stemming/Lemmatizing Definition	5
Index Construction	6
Term Weighting Definition	6
Index Design	6
Retrieval Model	7
Ranking	7
Implementation	8
Source Code	8
Video Demonstration	8
Discussion	9
References	10

List of Figures

List of Tables

Project Plan

Introduction

This project aims to build a search engine that allows students to retrieve relevant documents from a financial dataset. It is part of an educational initiative to promote financial literacy. This enables students to better understand and handle real-world financial situations encountered during their professional lives. To achieve this goal we have designed and implemented an *Information Retrieval (IR)* system that processes textual data, builds an *index* over a document collection, and *ranks* documents in response to user queries.

Team Roles and Responsibilities

Porter Clevidence

- Implemented BIM search
- Implemented BM25 search
- Implemented Language Model retrieval
- Implemented VSM search
- Search algorithm selection

Antonio Duran-Ramos

- Development environment setup
- Graphical interface design
- Written report
- Figures and Tables

Parth Gajjar

Arpit Vora

Methods and Techniques

To ensure cross-platform reproducibility across all of our team members, we used **Docker** for containerization. Docker allowed us to standardize our development environment so that our search engine behaved identical, regardless of the host machine's underlying operating system.

Within this Docker container, our project formalizes its Python dependencies with **uv** as the primary package and virtual environmental manager. **uv** ensured exact reproducibility of the software state for implementation of the search engine.

Within **uv**, the core retrieval logic and presentation layers are constructed with various Python libraries:

- **pandas** was used to parse the provided JSON dataset files to create structured data frames
- **nltk** and **scikit-learn** were used for natural language processing tasks; **nltk** for its implementation of the *Porter Stemming* algorithm and **scikit-learn** for its list of English stop words
- **scikit-learn** was also used for its functions **TfidfVectorizer** and **cosine_similarity**
- **rank-bm25** was used for its implementation of the BM25 ranking algorithm
- **streamlit** was used to implement the graphical interface of the search engine
- **matplotlib** was used to render figures and graphs based on retrieved data

Data

The dataset provided consisted of a collection of financial documents, along with static queries and their corresponding relevance judgments.

Data Description and Analysis

Dataset Statistics

The dataset consists of a collection of 5,000 financial text documents, 25 static queries, and corresponding relevance judgments. This dataset was provided in separate JSON files.

Text Processing

Surface Level Normalization Definition

Before tokenizing and any further text processing, the text undergoes a three-stage cleaning process to ensure variations in formatting do not create duplicate tokens in our index.

1. The `.lower()` function converts every character in the documents to lowercase.
2. We strip out any punctuation, including periods, commas, quotes, brackets, and special symbols using a regular expression. Specifically, `re.sub(r'[\W\s]', '', ...)`, which tells Python to find any character that is *not* a word character (numbers included) or a whitespace character, and replace it with nothing.
3. We again use a regular expression to target any sequence of one or more spaces to collapse them to a single, clean space. Additionally, we apply the `.strip()` function to this regular expression to trim any lingering spaces at the beginning and end of a document.

Tokenizer Definition

Now that the text has been normalized, we call `.split()` to break the strings apart at every single space. This function call results in a structured list of tokens.

Stopping Definition

At the same time the `.split()` function is tokenizing the text, we immediately check the current token against the `ENGLISH_STOP_WORDS` list provided by the `scikit-learn` library. If the token is a highly common word, (i.e., "the", "is", "at", etc.), the token is instantly discarded. This results in a final tokens list full of meaningful words.

Stemming/Lemmatizing Definition

Finally, the text processing pipeline ends by looping through the tokens and it applies NLTK's

`PorterStemmer()`, which removes common English suffixes and prefixes. This process shrinks our vocabulary drastically.

Index Construction

Term Weighting Definition

Index Design

Retrieval Model

Ranking

Implementation

Source Code

Video Demonstration

Discussion

References

[1]

[2]

[3]